Applied Linear Algebra
Unit 3

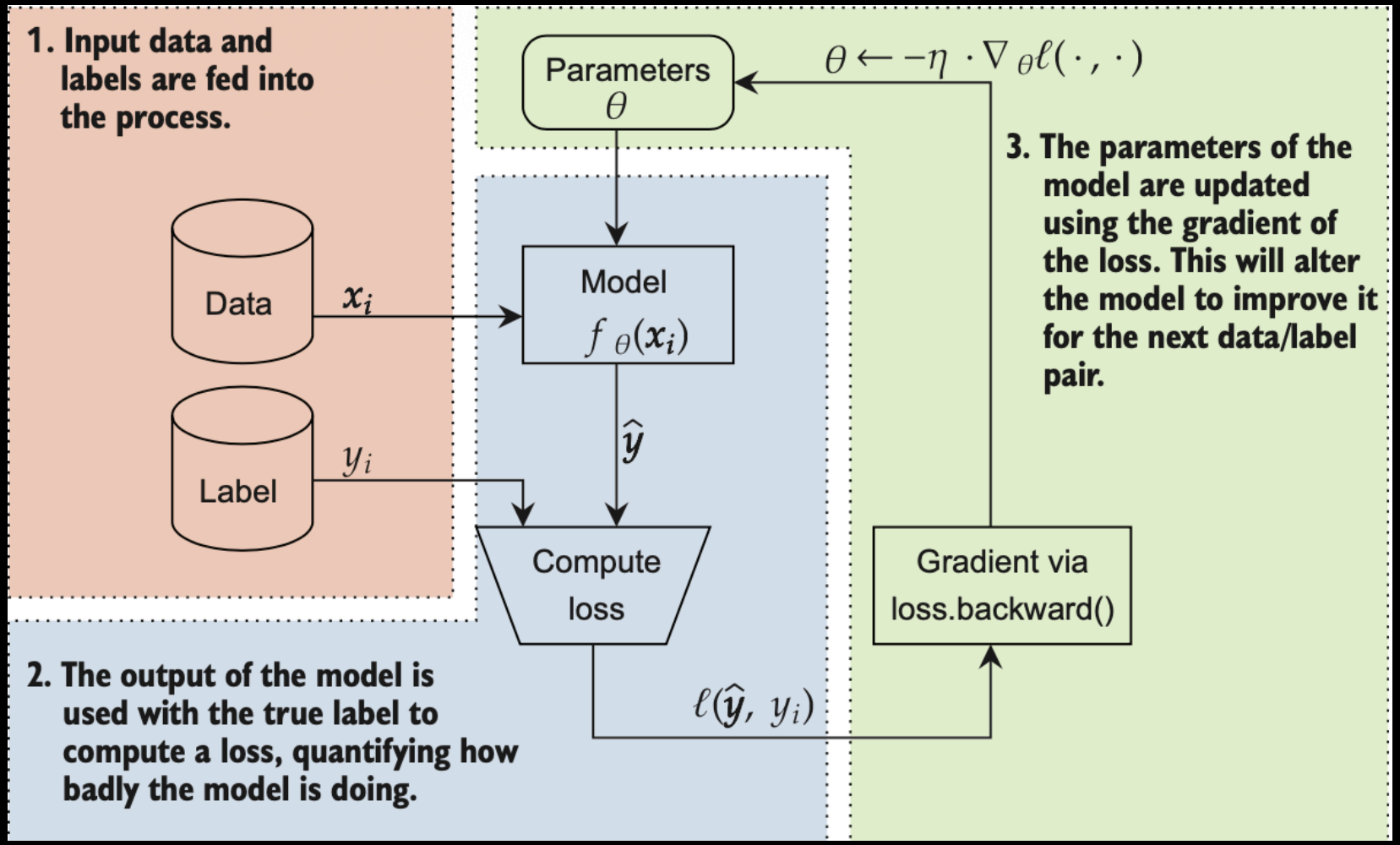
Devi Prasad M
Manipal School of Information Sciences
Manipal

October 2026

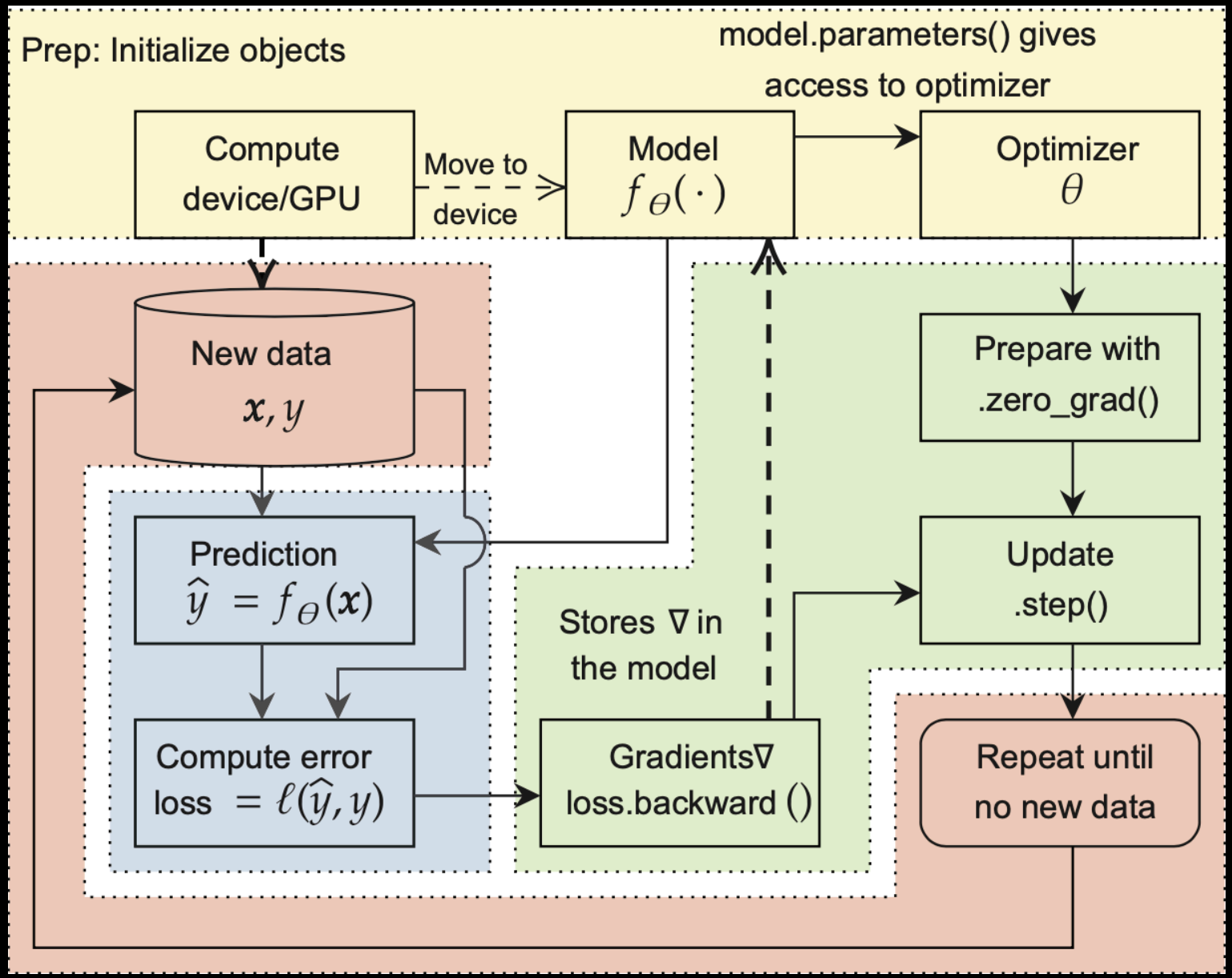
Topics and Time Allocation

- Derivatives in one dimension (0.5 h)
- Gradients in multiple dimensions (1 h)
- Chain rule in multiple dimensions (1 h)
- Gradients of matrix expressions (2 h)
- Optimization using gradient descent (3.5 h)
  - single-sample updates
  - Logistic regression (No hidden layer)
  - Softmax regression (No hidden layer)
- Backpropagation (3 h)
  - Computational graphs
  - Multi-layer perceptrons (Hidden Layers)

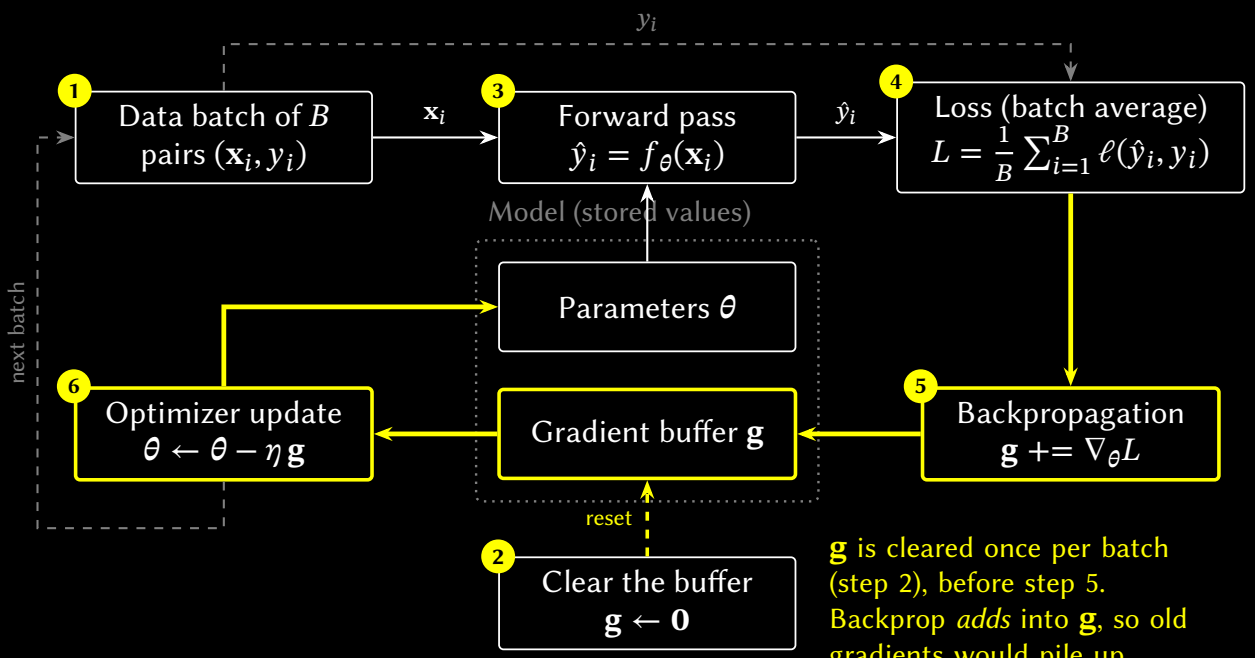
Training a Network Using PyTorch



Training a Network Using PyTorch

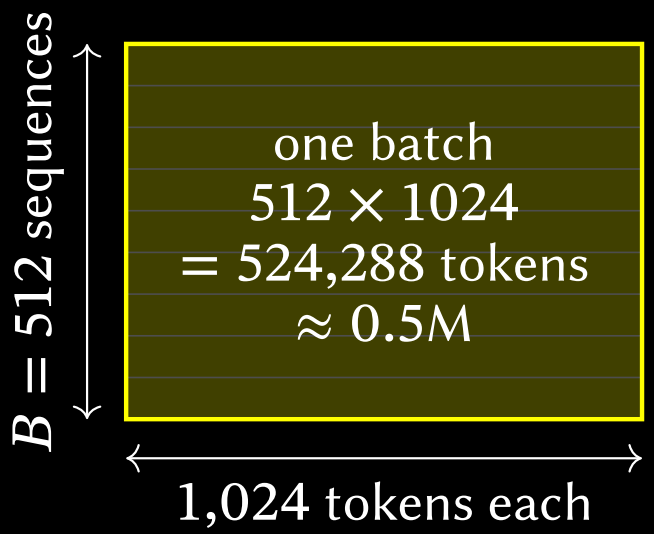


Where gradients are computed and used



In an LLM: a batch is B token sequences; steps 3–5 run once per batch, not once per sample, and give one gradient entry per parameter, billions in all.

GPT-2: one batch in numbers



Parameters	Layers	d_{model}
117M	12	768
345M	24	1024
762M	36	1280
1542M	48	1600

- "a larger batchsize of 512 is used"; context raised from 512 to 1,024 tokens.
- Vocabulary 50,257; WebText: about 8M documents, 40 GB.
- Learning rate "manually tuned" for each model.

Radford et al. (2019), Sec. 2.3 and Table 2. Total training tokens are not stated in the paper.

In an LLM: one forward pass, one backward pass and one update $\theta \leftarrow \theta - \eta\,\mathbf{g}$ per batch of about 0.5M tokens.

GPT-3: the batch grows with the model

Parameters	Layers	d_{model}	Batch (tokens)	Peak LR	Updates ≈
125M	12	768	0.5M	6.0×10^{-4}	600,000
350M	24	1024	0.5M	3.0×10^{-4}	600,000
760M	24	1536	0.5M	2.5×10^{-4}	600,000
1.3B	24	2048	1M	2.0×10^{-4}	300,000
2.7B	32	2560	1M	1.6×10^{-4}	300,000
6.7B	32	4096	2M	1.2×10^{-4}	150,000
13B	40	5140	2M	1.0×10^{-4}	150,000
175B	96	12288	3.2M	0.6×10^{-4}	94,000

Context 2,048 tokens; 300B training tokens for every model.
Updates ≈ tokens / batch.
Larger model ⇒ larger batch, *smaller* learning rate: the paper notes this trend.

Brown et al. (2020), Table 2.1. The "Updates" column is derived.

In an LLM: at 175B, a batch of 3.2M tokens is about 1,560 sequences, and training is only about 94,000 updates.

Llama 2 and Llama 3: Millions of Tokens per Update

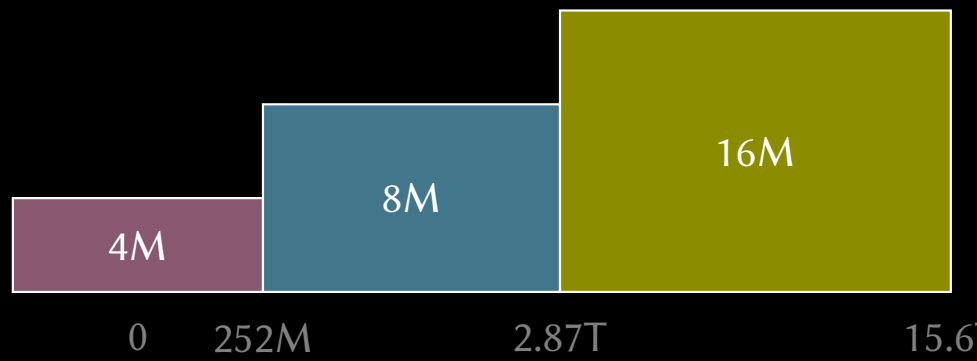
Llama 2 (7B to 70B)

- Global batch 4M tokens, context 4,096, 2T tokens in all: about 500,000 updates.
- AdamW, cosine schedule, 2,000 warmup steps, gradient clipping 1.0.
- Peak LR 3×10^{-4} (7B, 13B) and 1.5×10^{-4} (34B, 70B).

Llama 3

405B: the batch is ramped up

batch (tokens), sequences 4,096 → 8,192; not to sca

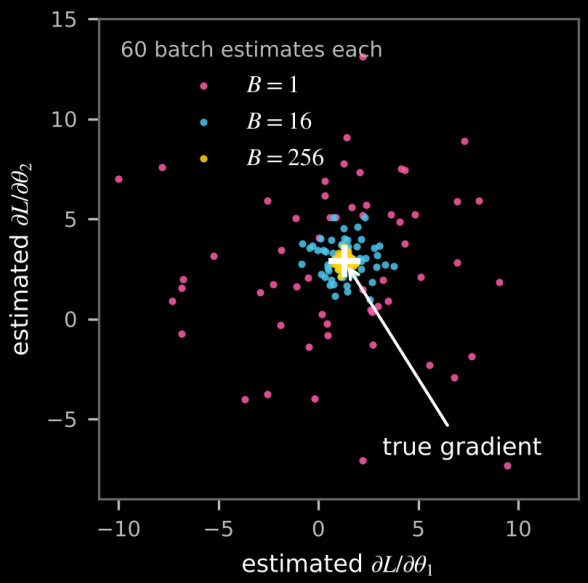


- AdamW, peak LR 8×10^{-5}, 8,000 warmup steps, cosine decay to 8×10^{-7}.

Touvron et al. (2023), Sec. 2.2; Llama Team (2024), Sec. 3.4.1. The Llama 3 paper says "8M sequences of 8,192 tokens"; read here as 8M tokens.

In an LLM: the batch size is not fixed: it is raised during training, alongside the learning-rate schedule.

Why the batch size is a hyperparameter



- **Gradient quality.** $\mathbf{g}$ is an average over B samples. The scatter shrinks roughly like $1/\sqrt{B}$ (synthetic example, same true gradient).
- **Learning rate.** The best η depends on B; GPT-3 pairs larger batches with smaller learning rates.
- **Cost.** Memory and hardware grow with B.
- **Number of updates.** With a fixed token budget, updates $\approx$ tokens / B.
- **Does not fit?** Split the batch into micro-batches and keep $\mathbf{g}$ (do not clear it) until the last one: gradient accumulation.

Learning is optimization in high dimensions

A deep network has N adjustable parameters θ (billions in an LLM). A loss $L(\theta)$ scores it on training data; training finds θ with small L by going downhill in $\mathbb{R}^N$.

$L(\theta)$ (that is f in the pictures) is one number computed from all N parameters

The picture: vary θ_i, θ_j (x, y), freeze the other $N-2$



→ a surface $f(x, y)$ we can draw

Partial derivative: vary only θ_i (x), freeze the other $N-1$



→ one curve, one slope $\partial L/\partial\theta_i$

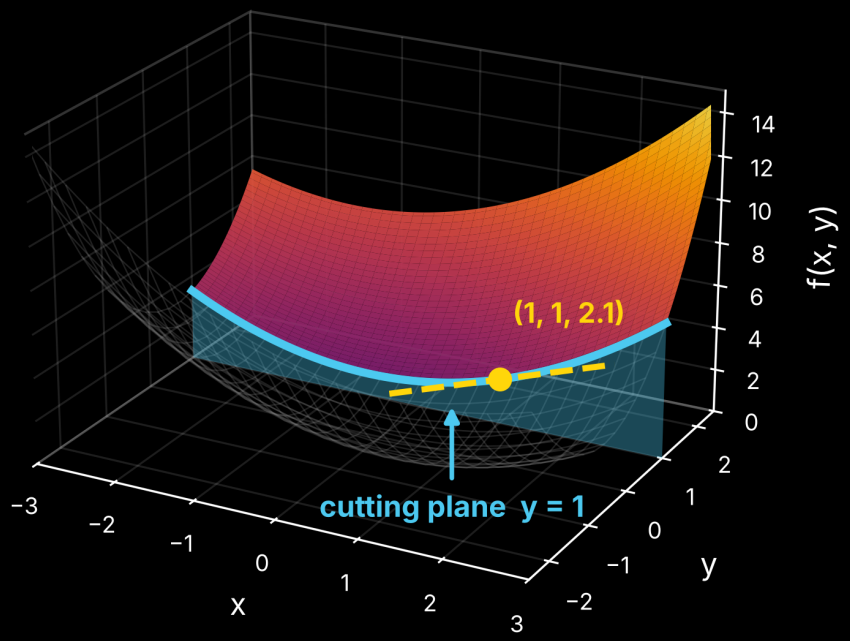
Gradient and training step: all N parameters at once



→ ∇L by backpropagation (next unit): N entries; then update $\theta \leftarrow \theta - \eta\nabla L$

Pictures vs. reality: our pictures show two parameters at a time; every definition that follows works for all N.

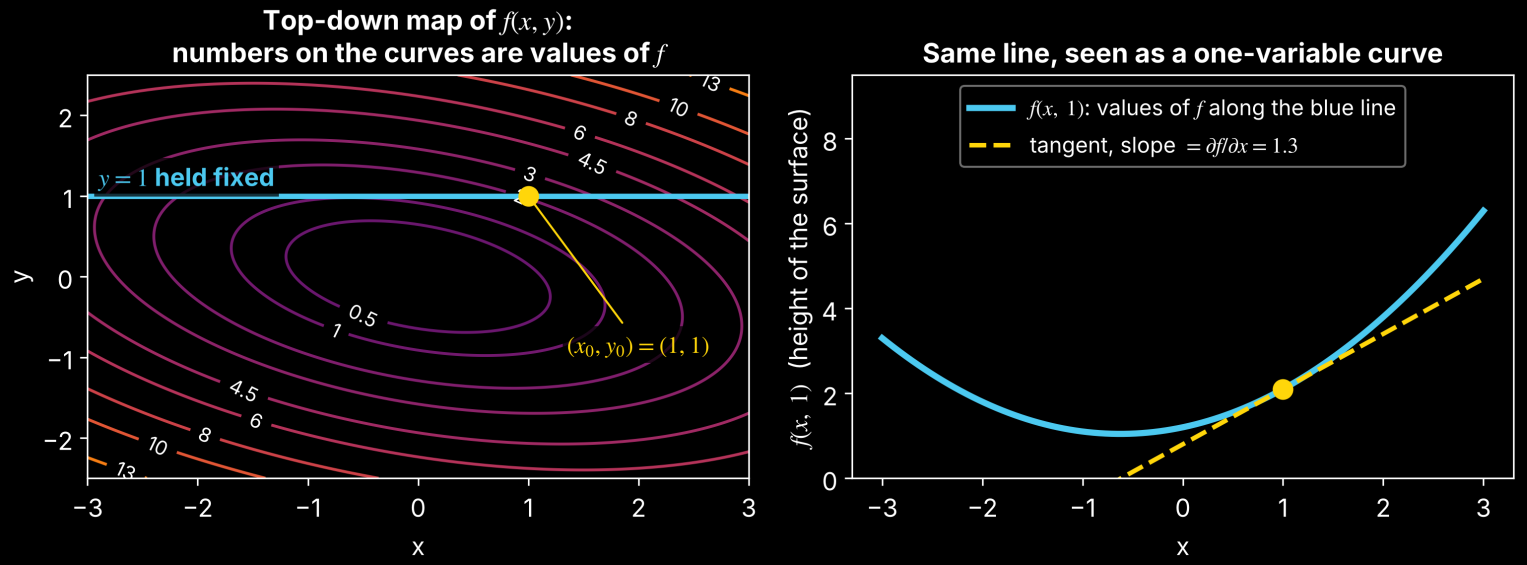
Slicing the loss: freeze all but one parameter



- A surface needs two inputs, so the picture varies x and y and freezes the rest.
- Cut it with the plane $y = 1$: only x varies, and what is left is an ordinary curve.
- Its slope at the yellow point is the partial derivative $\partial f / \partial x$.

In an LLM: pick one weight out of billions, freeze all the others and vary it: the loss traces a curve.

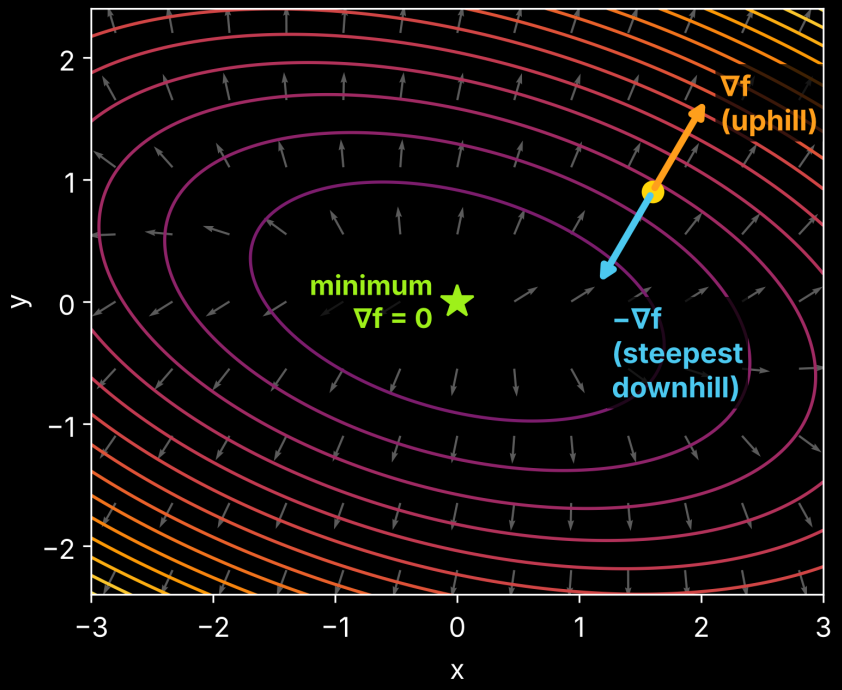
The partial derivative



$$\frac{\partial f}{\partial x_j}(\mathbf{x}) = \lim_{h \to 0} \frac{f(\mathbf{x} + h\,\mathbf{e}_j) - f(\mathbf{x})}{h}$$: change input j by a little and hold all the others fixed.

In an LLM: $\partial L / \partial \theta_j$ is the sensitivity of the loss to one weight.

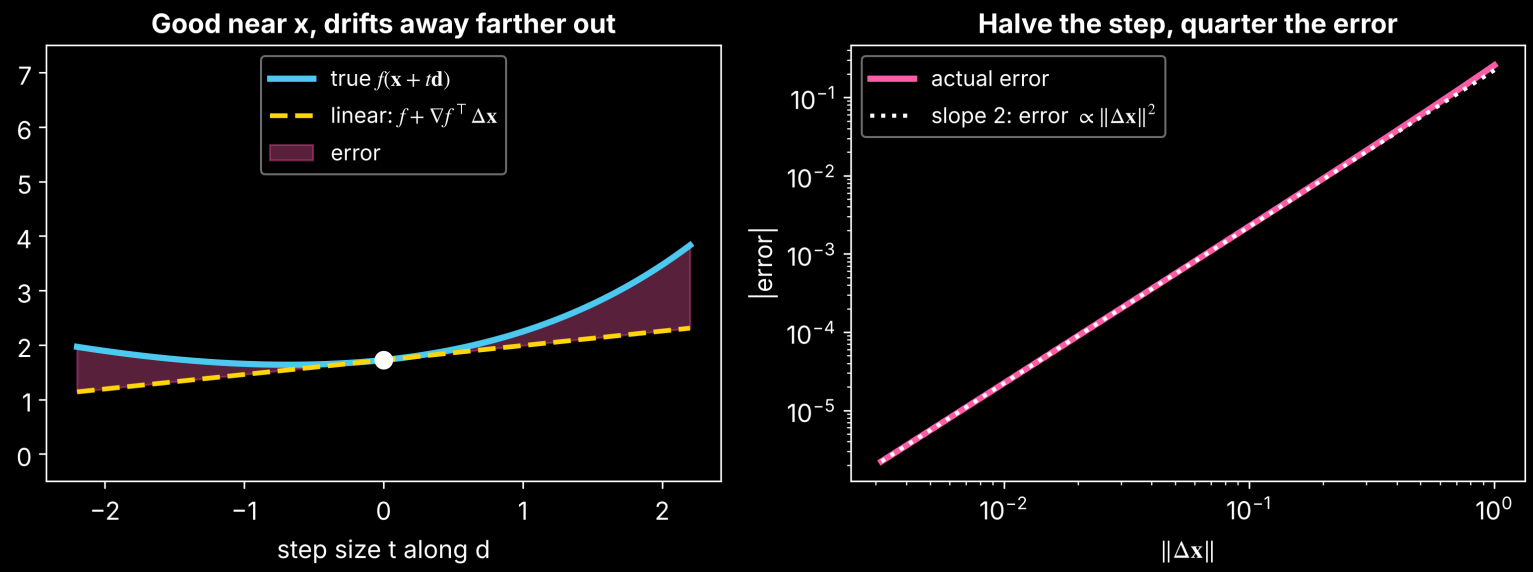
The gradient



- Collect the partials into one vector with the same shape as the input: $\nabla f(\mathbf{x}) = [\partial f/\partial x_1, \ldots, \partial f/\partial x_n]^\mathsf{T}$.
- ∇f points in the direction of fastest increase and is perpendicular to the contour lines.
- So $-\nabla f$ is the steepest way downhill.

In an LLM: ∇L has one entry per parameter, billions in all; $-\nabla L$ is the direction of steepest decrease.

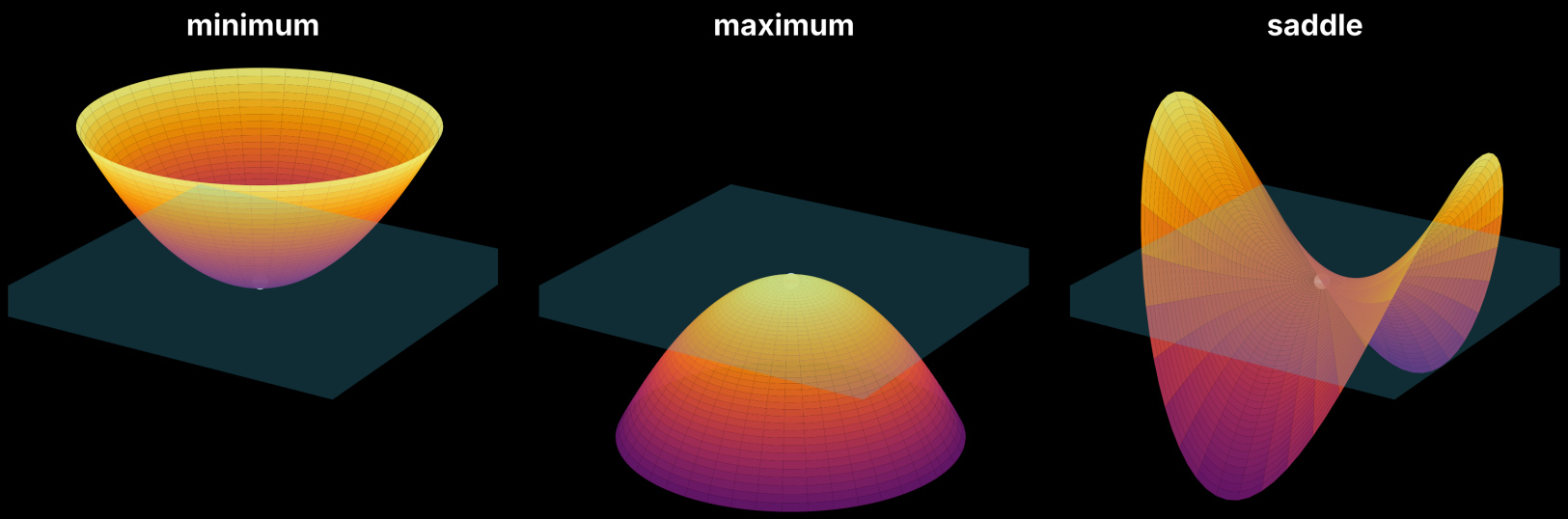
Local linearity and the downhill step



$f(\mathbf{x}+\Delta\mathbf{x}) \approx f(\mathbf{x}) + \nabla f(\mathbf{x})^{\intercal}\Delta\mathbf{x}$. Take $\Delta\mathbf{x} = -\eta\,\nabla f$: then $\Delta f \approx -\eta\,\|\nabla f\|^2$, a decrease for small η.

In an LLM: $\theta \leftarrow \theta - \eta\nabla L$ lowers L when the step is small enough; in practice ∇L is estimated on a batch of text.

A zero gradient does not certify a minimum



At any minimum, maximum or saddle, $\nabla f = \mathbf{0}$; the Hessian (coming later) tells them apart. The cyan plane is the flat tangent plane at the critical point.

In an LLM: training stops when the compute budget runs out, not at an exact minimum; in high dimensions, saddles are plausibly far more common than local minima.

Gradients in Multiple Dimensions

- **Partial derivative.** $\partial f / \partial x_j$: change one input, hold all the others fixed.
- **Gradient.** Collect the partials into a vector with the same shape as the input: $\nabla f(\mathbf{x}) = \left[\partial f / \partial x_1, \dots, \partial f / \partial x_n\right]^\top$.
- **Local linearity.** $f(\mathbf{x} + \Delta\mathbf{x}) \approx f(\mathbf{x}) + \nabla f(\mathbf{x})^\top \Delta\mathbf{x}$.
- **Steepest ascent.** ∇f points in the direction of fastest increase and is perpendicular to the contour lines, so $-\nabla f$ is the steepest way downhill. At a minimum, $\nabla f = \mathbf{0}$.
- **Jacobian.** For $\mathbf{f} : \mathbb{R}^n \to \mathbb{R}^m$, the $m \times n$ matrix with entries $J_{ij} = \partial f_i / \partial x_j$ records how every output responds to every input.

Chain Rule in Multiple Dimensions

- **Change flows through intermediate variables.** If $\mathbf{x} \to \mathbf{y} = \mathbf{g}(\mathbf{x}) \to z = f(\mathbf{y})$, a change in x_j reaches z through every y_i that depends on x_j.
- **Sum over paths.** $\dfrac{\partial z}{\partial x_j} = \displaystyle\sum_i \frac{\partial z}{\partial y_i}\,\frac{\partial y_i}{\partial x_j}$: multiply along each path, then add across paths.
- **Matrix form.** Jacobians multiply: $\mathbf{J}_{\mathbf{f}\circ\mathbf{g}}(\mathbf{x}) = \mathbf{J}_{\mathbf{f}}\big(\mathbf{g}(\mathbf{x})\big)\,\mathbf{J}_{\mathbf{g}}(\mathbf{x})$.
- **Scalar output.** When the final output is one number (a loss), $\nabla_{\mathbf{x}} z = \mathbf{J}_{\mathbf{g}}(\mathbf{x})^{\top} \nabla_{\mathbf{y}} z$: the gradient is passed backward through the transposed Jacobian.

Optimization Using Gradient Descent

- **Learning as minimization.** Training means choosing parameters θ that minimize the average loss $L(\theta) = \frac{1}{N} \sum_{n=1}^{N} \ell_n(\theta)$.
- **Update rule.** $\theta \leftarrow \theta - \eta \nabla L(\theta)$. By local linearity, a small step along $-\nabla L$ lowers the loss. If η is too small, progress is slow; if too large, the steps overshoot or diverge.
- **Single-sample updates.** $\theta \leftarrow \theta - \eta \nabla \ell_n(\theta)$ for a randomly chosen example n. Each step is cheap, and the gradient is noisy but correct on average.

Optimization Using Gradient Descent

- **Logistic regression.** $p = \sigma(\mathbf{w}^\top \mathbf{x} + b)$ with cross-entropy loss gives $\nabla_{\mathbf{w}} \ell = (p - y)\,\mathbf{x}$: prediction error times input.
- **Softmax regression.** $\mathbf{p} = \mathrm{softmax}(\mathbf{W}\mathbf{x} + \mathbf{b})$ with cross-entropy loss gives $\nabla_{\mathbf{W}} \ell = (\mathbf{p} - \mathbf{y})\,\mathbf{x}^\top$ for one-hot $\mathbf{y}$, the same pattern extended to K classes. Both losses are convex, so every local minimum is a global one.

Backpropagation

- **Computational graph.** The model and loss are written as a chain of simple operations. Each node needs only its own local derivative.
- **Forward, then backward.** The forward pass computes and stores every intermediate value. The backward pass applies the chain rule from the loss toward the inputs, reusing those stored values.
- **Error signals.** Each node receives $\partial L/\partial(\text{node})$ from the nodes it feeds. When a value feeds several nodes, the incoming signals add.
- **Efficiency.** One backward pass gives the gradient for every parameter, at a cost comparable to one forward pass.

Backpropagation

- **Multi-layer perceptron.** With $\mathbf{z}_1 = \mathbf{W}_1\mathbf{x} + \mathbf{b}_1$, $\mathbf{h} = \phi(\mathbf{z}_1)$, $\mathbf{p} = \mathrm{softmax}(\mathbf{W}_2\mathbf{h} + \mathbf{b}_2)$:

$$\boldsymbol{\delta}_2 = \mathbf{p} - \mathbf{y},$$

$$\nabla_{\mathbf{W}_2} L = \boldsymbol{\delta}_2 \mathbf{h}^\top,$$

$$\boldsymbol{\delta}_1 = (\mathbf{W}_2^\top \boldsymbol{\delta}_2) \odot \phi'(\mathbf{z}_1),$$

$$\nabla_{\mathbf{W}_1} L = \boldsymbol{\delta}_1 \mathbf{x}^\top$$

- **Non-convexity.** Hidden layers make the loss non-convex. Gradient descent reaches a local solution, and that solution depends on the initialization.

Reference Books

- Michael Nielsen, *Neural Networks and Deep Learning*
  http://neuralnetworksanddeeplearning.com/
- François Chollet, *Deep Learning with Python*,
  https://www.manning.com/books/deep-learning-with-python
- 3Blue1Brown
  https://www.3blue1brown.com/?topic=calculus